

03 — Scan Types

"Knowing when each scan type is chosen is knowing how the planner thinks."

Table of Contents

- 1. [Learning Objectives](#)
- 2. [Overview of All Scan Types](#)
- 3. [Sequential Scan \(Seq Scan\)](#)
- 4. [Index Scan](#)
- 5. [Index Only Scan](#)
- 6. [Bitmap Index Scan + Bitmap Heap Scan](#)
- 7. [TID Scan](#)
- 8. [ASCII Diagram: Scan Type Decision Flowchart](#)
- 9. [ASCII Diagram: Bitmap Scan Mechanics](#)
- 10. [Cost Model for Each Scan Type](#)
- 11. [Choosing Between Scan Types](#)
- 12. [Parallel Scans](#)
- 13. [EXPLAIN Output for Each Scan Type](#)
- 14. [Common Mistakes](#)
- 15. [Best Practices](#)
- 16. [Performance Considerations](#)
- 17. [Interview Questions & Answers](#)
- 18. [Exercises with Solutions](#)
- 19. [Production Scenarios](#)
- 20. [Cross-References](#)

Learning Objectives

After completing this section you will be able to:

- Describe each scan type and when the planner chooses it
- Explain the cost model that drives scan type selection
- Interpret EXPLAIN output for each scan type
- Identify when a scan type choice is suboptimal
- Force specific scan types for testing (not production use)

Overview of All Scan Types

Scan Type	When Used	I/O Pattern	Index Required?
Seq Scan	Full table, or index wouldn't help	Sequential	No
Index Scan	Selective filter, few matching rows	Random	Yes
Index Only Scan	All data in index, visibility OK	Random (index only)	Yes (covering)

Bitmap Index Scan	Many rows from one index	Batch	Yes
Bitmap Heap Scan	Reads heap pages identified by Bitmap Index Scan	Near-sequential	Yes (via Bitmap)
TID Scan	Query by ctid (system column)	Random	No
Parallel Seq Scan	Full table, parallel workers	Parallel sequential	No
Parallel Index Scan	Parallel index traversal	Parallel random	Yes

Sequential Scan (Seq Scan)

What It Does

Reads every page of the table from beginning to end, examining each tuple against the WHERE predicate.

```
Heap file:
Page 0 → Page 1 → Page 2 → ... → Page N
[read every tuple, check predicate, return matches]
```

When Chosen

- No suitable index exists
- Selectivity is high (query matches many rows, e.g., >5-10%)
- Table is very small (< 8-16 pages, seq scan is always fastest)
- Statistics indicate most rows will match

EXPLAIN Output

```
EXPLAIN SELECT * FROM orders WHERE status = 'completed';
-- (where 80% of rows have status='completed')
```

```
Seq Scan on orders (cost=0.00..45678.00 rows=8000000 width=128)
  Filter: (status = 'completed')
```

Parallel Seq Scan

For large tables PostgreSQL can split the sequential scan across multiple workers:

```
Gather (cost=1000.00..23456.00 rows=8000000 width=128)
  Workers Planned: 4
  -> Parallel Seq Scan on orders (cost=0.00..23456.00 rows=2000000 width=128)
    Filter: (status = 'completed')
```

Index Scan

What It Does

1. Traverses the B-tree (or other index) to find entries matching the predicate
2. For each matching index entry, fetches the corresponding heap tuple (random I/O)
3. Returns matching tuples

```
Index (sorted by customer_id):  
[Navigate to customer_id=42, find ctids]  
↓  
Heap (random page access for each ctid):  
[Fetch pages containing customer 42's rows]
```

When Chosen

- Highly selective predicate (few matching rows)
- Index exists on the predicate column(s)
- `random_page_cost` is low relative to the number of heap pages to fetch

EXPLAIN Output

```
EXPLAIN SELECT * FROM orders WHERE customer_id = 42;
```

```
Index Scan using idx_orders_customer on orders  
  (cost=0.56..8.58 rows=5 width=128)  
Index Cond: (customer_id = 42)
```

With Buffers

```
Index Scan using idx_orders_customer on orders  
  (actual time=0.034..1.234 rows=50 loops=1)  
Index Cond: (customer_id = 42)  
Buffers: shared hit=8 read=3  
-- 8 pages from cache, 3 from disk  
-- Total: 11 pages (3 index + 8 heap)
```

Index Only Scan

What It Does

Like an Index Scan, but the required data is entirely within the index (covering index). Heap pages are accessed ONLY to check the visibility map.

```
Covering Index (customer_id key + email, created_at as INCLUDE):  
[Navigate index, find matching entries, read data from index leaf pages]  
↓  
Visibility Map check (lightweight, 1 bit per heap page):  
[Confirm page is all-visible → no heap read needed]
```

When Chosen

- All SELECT columns are in the index (as key or INCLUDE columns)
- The visibility map confirms heap pages are all-visible
- After VACUUM has run on the table

EXPLAIN Output

```
CREATE INDEX ON users(user_id) INCLUDE (email, created_at);
VACUUM users;
EXPLAIN (ANALYZE, BUFFERS) SELECT email, created_at FROM users WHERE user_id = 42;
```

```
Index Only Scan using idx_users_covering on users
  (cost=0.43..4.45 rows=1 width=40)
  (actual time=0.023..0.023 rows=1 loops=1)
Index Cond: (user_id = 42)
Heap Fetches: 0          ← no heap access!
Buffers: shared hit=3
```

Heap Fetches: 0 confirms zero heap reads — pure index-only scan.

When Heap Fetches > 0

```
Index Only Scan (actual time=...)
Heap Fetches: 1234 ← visibility check failed, heap pages needed
```

Run `VACUUM table_name` to set all-visible bits.

Bitmap Index Scan + Bitmap Heap Scan

What It Does

A two-phase scan:

Phase 1: Bitmap Index Scan

- Traverses the index and collects ALL matching ctids
- Creates an in-memory bitmap: each bit = one heap page (or tuple, if exact)
- Does NOT access heap pages yet

Phase 2: Bitmap Heap Scan

- Reads heap pages identified in the bitmap, in page-number order
- For each page, applies any "Recheck" conditions
- Returns matching tuples

```
Phase 1 (Bitmap Index Scan):
Index traversal → collect ctids → build bitmap

Bitmap:
Page 0: 0 (no matching rows)
Page 1: 1 (has matching rows)
Page 2: 0
Page 3: 1
```

...

Page N: 1

Phase 2 (Bitmap Heap Scan):

Read pages in order: 1, 3, ..., N (sequential-ish)

Apply predicates to individual tuples on each page

Why Bitmap?

The bitmap allows heap pages to be read in **sorted order** (sequential-ish), avoiding the random I/O problem of Index Scan when many rows match.

Multiple Bitmap Index Scans (BitmapAnd / BitmapOr)

```
-- Two indexes: idx_customer and idx_status
EXPLAIN SELECT * FROM orders WHERE customer_id = 42 AND status = 'pending';
```

Bitmap Heap Scan on orders

Recheck Cond: ((customer_id = 42) AND (status = 'pending'))

-> BitmapAnd

-> Bitmap Index Scan on idx_orders_customer

Index Cond: (customer_id = 42)

-> Bitmap Index Scan on idx_orders_status

Index Cond: (status = 'pending')

BitmapAnd intersects two bitmaps; BitmapOr unions them.

Lossy Bitmap (BRIN or very large result sets)

Bitmap Heap Scan on events

Recheck Cond: (created_at > '2024-01-01')

Heap Blocks: lossy=2345 ← lossy: entire block ranges

-> Bitmap Index Scan on idx_events_brin ← BRIN produces lossy bitmaps

"Lossy" means the bitmap represents entire page groups (not individual tuples), requiring a full Recheck of each page.

TID Scan

Accesses rows directly by ctid (physical location: page + offset).

```
SELECT * FROM orders WHERE ctid = '(0,1)';
```

TID Scan on orders (cost=0.00..4.01 rows=1 width=128)

TID Cond: (ctid = '(0,1)::tid)

Rarely used in application queries. Useful for:

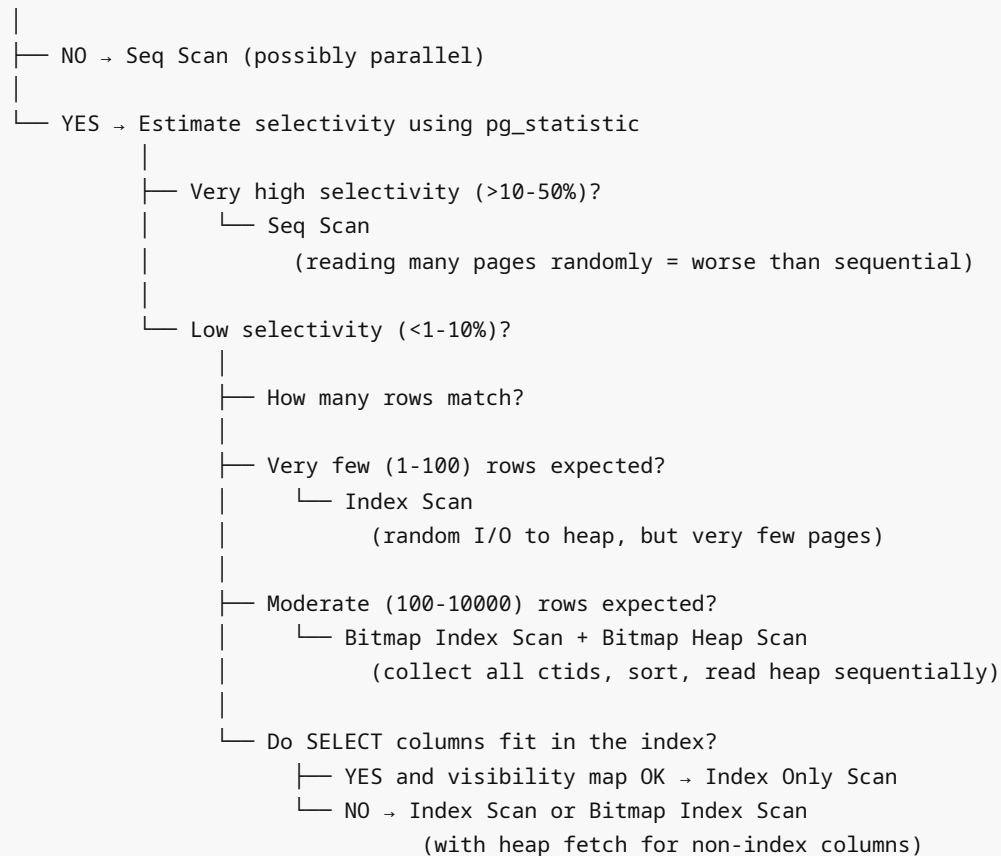
- Direct low-level row access (debugging)
- `WHERE ctid = ANY(ARRAY[...])` after collecting ctids externally

- Batch processing by ctid range

ASCII Diagram: Scan Type Decision Flowchart

Query: SELECT ... FROM table WHERE condition

Is there an index on the predicate column(s)?



ASCII Diagram: Bitmap Scan Mechanics

Table: orders (10,000 pages)

Index: orders(customer_id)

Query: WHERE customer_id BETWEEN 100 AND 200 (matching ~500 rows on ~200 pages)

Phase 1: Bitmap Index Scan

```
| Index leaf pages containing customer_id 100-200: |
|   ctid=(1,5), ctid=(1,8), ctid=(4,2), ctid=(4,9), |
|   ctid=(7,1), ctid=(8,3), ctid=(8,4), ... |
```

|
▼ Build bitmap sorted by heap page

```
| Bitmap (1 bit per heap page): |
```

Page 0: 0	Page 1: 1	Page 2: 0	Page 3: 0	Page 4: 1	
Page 5: 0	Page 6: 0	Page 7: 1	Page 8: 1	Page 9: 0	...

▼ Phase 2: Bitmap Heap Scan (pages in order)

Read page 1 → check tuple 5 and 8 → return matching
Read page 4 → check tuple 2 and 9 → return matching
Read page 7 → check tuple 1 → return matching
Read page 8 → check tuples 3 and 4 → return matching
Pages read: 1, 4, 7, 8 (4 pages in order, not 500 random)
vs Index Scan: might read 500 random pages (if rows scattered)

Memory usage: $\sim 10,000 \text{ bits} / 8 = \sim 1.25 \text{ KB}$ for bitmap

If work_mem is very low, bitmap becomes "lossy" (1 bit per page group instead of per page)

Cost Model for Each Scan Type

Sequential Scan Cost

$$\begin{aligned} \text{cost} &= \text{pages} \times \text{seq_page_cost} + \text{rows} \times \text{cpu_tuple_cost} \\ &= \text{pages} \times 1.0 + \text{rows} \times 0.01 \end{aligned}$$

Index Scan Cost

$$\begin{aligned} \text{cost_startup} &= \text{index_height} \times \text{random_page_cost} \\ \text{cost_total} &= \text{index_pages} \times \text{random_page_cost} + \text{matching_rows} \times \text{random_page_cost} + \\ &\quad \text{matching_rows} \times \text{cpu_tuple_cost} \end{aligned}$$

Index Only Scan Cost

$$\begin{aligned} \text{cost_total} &= \text{index_pages} \times \text{random_page_cost} + \text{heap_fetches} \times \text{random_page_cost} \\ &\quad (\text{heap_fetches} \ll \text{matching_rows} \text{ when visibility map is set}) \end{aligned}$$

Bitmap Index Scan + Heap Scan Cost

$$\begin{aligned} \text{cost_bitmap_index} &= \text{index_pages} \times \text{random_page_cost} \\ \text{cost_bitmap_heap} &= \text{matching_heap_pages} \times (\text{seq_page_cost} + \text{random_page_cost} \times 0.5) \\ &\quad + \text{rows} \times \text{cpu_tuple_cost} \\ &\quad (\text{heap pages read in near-sequential order}) \end{aligned}$$

Breakeven Points (approximate, defaults)

- Seq Scan vs Index Scan: $\sim 1\text{-}5\%$ selectivity
- Index Scan vs Bitmap Scan: $\sim 10\text{-}100$ matching rows
- On SSD (random_page_cost=1.1): Index Scan competitive to $\sim 20\text{-}30\%$ selectivity

Choosing Between Scan Types

The planner considers these factors:

1. **Selectivity**: lower → more likely to use index
2. **Table size**: larger → seq scan more expensive → index preferred earlier
3. **random_page_cost** vs **seq_page_cost**: higher ratio → prefer seq scan; lower ratio (SSD) → prefer index
4. **effective_cache_size**: more cache → index pages likely cached → index scan cheaper
5. **Work_mem**: more → bitmap bitmap fits in memory → bitmap scan more efficient

Forcing Scan Types (for testing only)

```
SET enable_seqscan = off;           -- force index usage
SET enable_indexscan = off;         -- force bitmap scan
SET enable_indexonlyscan = off;     -- force index scan over index-only
SET enable_bitmapscan = off;        -- force index scan instead of bitmap
-- Always reset after testing:
SET enable_seqscan = on;
```

Parallel Scans

PostgreSQL can parallelize both Seq Scan and certain Index Scans.

Configuration

```
-- Enable parallel query (default: on)
SET max_parallel_workers_per_gather = 4;
SET min_parallel_table_scan_size = '8MB'; -- tables larger than this get parallel scan
SET parallel_setup_cost = 1000;           -- overhead of launching parallel workers
SET parallel_tuple_cost = 0.1;           -- cost per tuple for parallel overhead
```

Parallel Seq Scan EXPLAIN

```
Gather  (cost=1000.00..23456.00 rows=1000000 width=64)
  Workers Planned: 4
    -> Parallel Seq Scan on orders
        (cost=0.00..5000.00 rows=250000 width=64)
        Filter: (status = 'completed')
```

Each worker scans 1/4 of the table; Gather collects and returns results.

When Parallel Scans Are Used

- Table is larger than `min_parallel_table_scan_size`
- Query doesn't require ordering (or Gather Merge is used)
- `max_parallel_workers_per_gather > 0`

- No parallelism-unsafe operations in the query

EXPLAIN Output for Each Scan Type

Seq Scan

```
Seq Scan on orders  (cost=0.00..45678.00 rows=1000000 width=128)
                    (actual time=0.023..1234.567 rows=800000 loops=1)
  Filter: (status = 'completed')
 Rows Removed by Filter: 200000
 Buffers: shared hit=5000 read=1000
```

Index Scan

```
Index Scan using idx_orders_customer on orders
  (cost=0.56..8.58 rows=5 width=128)
  (actual time=0.034..0.567 rows=5 loops=1)
 Index Cond: (customer_id = 42)
 Buffers: shared hit=4
```

Index Only Scan

```
Index Only Scan using idx_users_covering on users
  (cost=0.43..4.45 rows=1 width=40)
  (actual time=0.023..0.023 rows=1 loops=1)
 Index Cond: (user_id = 42)
 Heap Fetches: 0
 Buffers: shared hit=3
```

Bitmap Index + Heap Scan

```
Bitmap Heap Scan on orders  (cost=23.45..856.12 rows=500 width=128)
                          (actual time=0.234..5.678 rows=487 loops=1)
 Recheck Cond: (customer_id BETWEEN 100 AND 200)
 Heap Blocks: exact=45
 Buffers: shared hit=52
->  Bitmap Index Scan on idx_orders_customer
      (cost=0.00..23.33 rows=500 width=0)
      (actual time=0.189..0.189 rows=487 loops=1)
      Index Cond: (customer_id BETWEEN 100 AND 200)
```

Common Mistakes

1. Assuming Index Scan is always better than Seq Scan

```
-- If this returns 80% of rows, Seq Scan is correct choice!
SELECT * FROM orders WHERE status = 'completed'; -- 80% of orders
```

```
-- Don't force an index here – it would be slower!
```

2. Not understanding when Bitmap Scan is the right choice

Bitmap Index Scan is not a fallback — it's the optimal choice for moderate selectivity where Index Scan would do too many random reads.

3. Ignoring Heap Blocks in Bitmap Scan output

```
Heap Blocks: exact=450      ← 450 heap pages with individual bit tracking (good)
Heap Blocks: lossy=2345     ← 2345 block ranges (less precise, BRIN or low work_mem)
```

`lossy` means `work_mem` was too small for an exact bitmap — increase `work_mem`.

4. Using `SET enable_seqscan = off` in production

This is a debugging tool only. In production, the planner may have good reasons for choosing a Seq Scan. Forcing index usage can actually make queries slower.

Best Practices

1. **Trust the planner** for scan type selection — it uses the cost model correctly when statistics and configuration are accurate.
 2. **Set `random_page_cost = 1.1`** on SSD storage — this alone can cause the planner to prefer index scans in many more situations.
 3. **Set `effective_cache_size`** to ~75% of RAM — more accurate index scan cost estimates.
 4. **Run ANALYZE regularly** — stale statistics cause wrong selectivity estimates, leading to wrong scan type choices.
 5. **Use covering indexes** to enable Index Only Scans for high-frequency queries.
 6. **Check `Heap Blocks: lossy`** — if present in critical queries, either increase `work_mem` or switch from BRIN to B-tree.
-

Performance Considerations

Seq Scan Optimization

```
-- Parallel seq scan for large tables
SET max_parallel_workers_per_gather = 4;
-- Or: table partitioning (each partition scanned in parallel or in order)
-- Or: add an index for the most selective predicate
```

Index Scan on Cold Data

For tables rarely read, index scan pages may not be in cache:

```
Index Scan (actual time=...)
  Buffers: shared hit=2 read=98 ← 98 disk reads!
```

This is normal for cold cache — the first few requests are slow, subsequent ones hit cache. Consider `pg_prewarm` for critical tables.

Bitmap Scan and work_mem

A lossy bitmap occurs when `work_mem` is insufficient for an exact bitmap (1 bit per tuple). The lossy bitmap uses 1 bit per page group, requiring more heap pages to be read.

```
SET work_mem = '64MB'; -- ensure exact bitmap for large result sets
```

Interview Questions & Answers

Q1. What is the difference between an Index Scan and a Bitmap Index Scan?

A: An Index Scan navigates the index, then immediately fetches each matching heap tuple in random order (one heap page read per row). It's optimal for very few matching rows because each random I/O is focused. A Bitmap Index Scan collects ALL matching ctids first, builds an in-memory bitmap sorted by heap page number, then reads heap pages in order via Bitmap Heap Scan. It's optimal for moderate numbers of matching rows because it converts random reads into near-sequential reads. The crossover between them is typically around 10-100 matching rows.

Q2. When would PostgreSQL choose a Seq Scan even when an index exists on the predicate column?

A: The planner chooses a Seq Scan when: (1) The query is expected to return a large fraction of rows (high selectivity) — typically >5-20% — making sequential I/O cheaper than many random reads. (2) The table is very small (< 8-16 pages) — seq scan is always fast for tiny tables. (3) `random_page_cost` is set too high (e.g., default 4.0 on SSDs where 1.1 is appropriate) — artificially inflating the cost of index scans. (4) `effective_cache_size` is set too low — the planner underestimates how much data is cached.

Q3. What does "Heap Blocks: lossy" mean in a Bitmap Heap Scan?

A: In a Bitmap Heap Scan, "exact" mode means the bitmap tracks which specific tuples on a page match. "Lossy" mode means `work_mem` was insufficient to track individual tuples, so the bitmap only records that a page MIGHT contain matching tuples (1 bit per page instead of per tuple). In lossy mode, the Bitmap Heap Scan must read the entire page and recheck each tuple against the predicate. The Recheck Cond in the plan confirms this. BRIN indexes always produce lossy bitmaps because they store block range summaries, not exact row locations.

Q4. Under what conditions does an Index Only Scan occur?

A: An Index Only Scan occurs when: (1) All columns referenced in the query (SELECT and WHERE) are available from the index (either as key columns or INCLUDE columns). (2) The visibility map for the corresponding heap pages is marked "all-visible" (set by VACUUM), confirming all tuples are visible to all transactions. If condition 2 fails for some pages, those pages are still fetched (heap fetches), but the scan is still labeled "Index Only Scan." Heap Fetches: 0 in EXPLAIN ANALYZE confirms a true heap-free scan.

Exercises with Solutions

Exercise 1

A table `transactions` has 100M rows. Query: `WHERE account_id = 12345 AND amount > 100`. An index exists on `(account_id, amount)`. Predict the scan type for each scenario:

- Scenario A: 3 rows match
- Scenario B: 50,000 rows match
- Scenario C: 8M rows match

Solution:

- Scenario A (3 rows): **Index Scan** — very few rows, minimal random I/O
 - Scenario B (50,000 rows / 0.05% selectivity): **Bitmap Index Scan + Bitmap Heap Scan** — too many for Index Scan, collects ctids and reads heap in order
 - Scenario C (8M rows / 8% selectivity): **Seq Scan** — reading 8% of pages randomly is more expensive than reading 100% sequentially
-

Production Scenarios

Scenario 1: Unexpected Seq Scan on SSD System

A query that should use an index was doing a Seq Scan on an SSD-based server.

```
-- Problem:
SHOW random_page_cost; -- shows 4.0 (HDD default!)
-- On SSDs, this makes index scans look 4× more expensive than they are

-- Fix:
ALTER SYSTEM SET random_page_cost = 1.1;
SELECT pg_reload_conf();
-- Now the planner correctly assesses index scan costs on SSDs
```

Cross-References

- [01_explain_basics.md](#) — Reading EXPLAIN output
- [02_explain_analyze.md](#) — EXPLAIN ANALYZE details
- [04_join_algorithms.md](#) — Join node types
- [../07_Indexes/01_index_fundamentals.md](#) — Index fundamentals
- [05_query_planner.md](#) — Planner cost model